



Anton Rx Challenge — Q&A for Hackathon Teams

Innovation Hacks 2.0 • April 3–5, 2026 • Arizona State University

1. Who should we treat as the primary user for this problem?

A market access analyst or formulary strategist at a company like Anton Rx. Someone whose job is to advise health plan clients on drug cost strategy. They need to quickly understand how a specific drug is covered across multiple payers — what the criteria are, how they differ, and what's changed recently. They're not a developer; they're a domain expert who currently does this work manually by reading PDFs.

2. What's the biggest pain point today when working with these policies?

There are a few layers to it. First, there's the discovery problem — analysts have to manually visit each payer's website to track down the relevant policies, and every payer organizes them differently (some have a searchable index, some bury them in a portal, some put everything in one massive PDF). Just finding the right document is time-consuming.

Then there's the extraction and comparison problem — once you have the policies, every plan uses different formats, structures, and terminology, so an analyst might spend hours answering one question like "Which plans require step therapy before approving Drug X?" because they have to open 10 different PDFs, find the relevant section in each one, and mentally normalize the information.

On top of all that, policies can change at any time — not just quarterly, but sometimes with little or no notice — so the work is never done. You're always chasing the latest version.

3. If we had to focus on one MVP feature, what would be most valuable?

Side-by-side comparison of coverage criteria for a single drug across multiple payers. If a user can type in a drug name and get a normalized table showing "here's what Cigna requires, here's what UHC requires, here's what BCBS NC requires" — that alone would be enormously valuable. That's the question analysts answer most often and it's the most painful to do manually.

A strong bonus feature would be the ability to compare a previous version of a policy against the newest version to surface what actually changed. Policies get updated regularly, but sometimes the changes are minor — a new clinical reference, a formatting tweak, an updated effective date — and sometimes they're significant, like a new step therapy requirement or a removed indication. Today, an analyst has to read both versions end-to-end to figure out which it is. A tool that could

highlight meaningful clinical or coverage changes versus cosmetic edits would save a huge amount of time.

4. What key data fields are most important to extract from policies?

In rough priority order: drug name (brand and generic), drug category (the therapeutic class or grouping the drug falls into), the drug's access status within that category (preferred vs. non-preferred, and how many drugs share that preferred status), covered indications/diagnoses, prior authorization requirements (yes/no and what criteria), step therapy requirements (what drugs must be tried first), site-of-care restrictions, dosing/quantity limits, and effective date of the policy.

The category and access status piece is particularly important because it directly drives rebate economics — a drug that's preferred 1 of 2 in its category commands a different rebate rate than a drug that's preferred 1 of 3 or has exclusive preferred status. Understanding a drug's position relative to the other drugs in its category is just as important as understanding the coverage criteria themselves. If you can reliably extract even the first five or six of those fields, that's a strong prototype.

5. How would you prioritize comparison vs Q&A vs change tracking?

Q&A and change tracking are tied for first priority. Being able to ask a plain-language question like “Does Cigna cover Rituxan for lupus?” or “What step therapy does UHC require for Humira?” and get a fast, accurate answer is the most immediate time saver — that's the kind of question analysts are answering all day long. Change tracking is equally important because knowing what changed in a policy (and whether it's a meaningful clinical change or just a cosmetic edit) is critical to staying current and advising clients correctly.

Cross-payer comparison — seeing how coverage for the same drug differs across multiple plans side by side — would be the second priority. It's extremely valuable, but it builds naturally on top of the first two: if you can reliably extract and understand individual policies, comparison becomes a presentation layer on top of that.

6. What would make a demo really stand out for this use case?

Three things. First, showing automated policy retrieval — demonstrating that the system can pull policies from multiple payer websites without someone having to manually visit each site, navigate their unique layout, and download the right document. That alone addresses one of the biggest time sinks in the current workflow.

Second, showing the system working across policies that look completely different from each other (e.g., a UHC single-drug PDF vs. a UPMC mega-document vs. an EmblemHealth portal download) — that proves it can handle real-world messiness, not just one format.

Third, a clean comparison view where a non-technical person can immediately see the differences between payers without reading raw policy text. If a judge can glance at the screen and instantly understand “oh, Cigna requires step therapy but UHC doesn't” — that's the wow moment.

7. Do we need to include user authentication (like Google login) for this project, or is that not necessary for the hackathon?

Not necessary at all — for the hackathon, we'd recommend skipping authentication and focusing your time on the core problem: ingesting policies, extracting structured data, and building a useful interface. The judges will be most impressed by what happens when someone types in a drug name, not by a login screen. Save your energy for the parts that demonstrate your solution.

8. Do you know what the judging criteria will focus on?

Yes — it's in the problem statement document, but here's the breakdown:

- **Problem Understanding (20%)** — Do you clearly understand the real-world pain point?
- **Technical Implementation (25%)** — How effectively does your solution use AI/ML, NLP, or other technical approaches?
- **Usability & Design (20%)** — Is the output clear, navigable, and useful to a non-technical stakeholder?
- **Completeness (20%)** — How end-to-end is the prototype? (ingestion → extraction → comparison → interface)
- **Creativity & Wow Factor (15%)** — Did you do something unexpected or particularly clever?

The biggest weight is on technical implementation, but usability is right behind it. A tool that extracts data brilliantly but is impossible to use won't score as well as one that balances both.